

MEMORIA DE LA PRÁCTICA

RUEDAFÁCIL

Sistema de Gestión de Alquiler de Vehículos

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Curso: 2025-2026

Sevilla, Mayo 2026

1. ESTUDIO DEL PROBLEMA Y ANÁLISIS DEL SISTEMA

Introducción

RuedaFácil es un programa informático creado para ayudar a una empresa de alquiler de vehículos a organizar su trabajo diario. Este programa se maneja escribiendo órdenes en una pantalla de texto (lo que se llama "aplicación de consola") y permite llevar el control de los clientes, los coches, las furgonetas, los empleados y los contratos de alquiler.

La idea principal es que la empresa pueda saber en todo momento qué vehículos tiene disponibles, quién los ha alquilado, qué empleado gestionó cada alquiler y cuál es el historial de cada cliente. Todo esto se guarda en una base de datos (un almacén de información organizado) para que no se pierdan los datos.

Funciones y rendimiento

El sistema permite gestionar:

- Clientes
- Vehículos (categorías y disponibilidad)
- Alquileres (creación, seguimiento e historial)
- Empleados
- Categorías

El programa debe responder con rapidez a las acciones del usuario (como dar de alta un cliente o consultar un alquiler), sin demoras molestas. Al tratarse de una aplicación de consola y trabajar con una base de datos ligera (MySQL), el rendimiento es bueno incluso en equipos sencillos.

Objetivos del sistema

- **Gestión de clientes:** Altas, bajas, modificaciones y búsquedas de clientes, identificados por su DNI. Se almacenan sus datos personales, contacto y número de carnet de conducir.
- **Gestión de vehículos y categorías:** Administración de la flota (CRUD), con clasificación por categorías (turismo, furgoneta, vehículo eléctrico, etc.). Se incluye la herencia para modelar vehículos eléctricos con autonomía específica.
- **Gestión de empleados:** Registro de los trabajadores que gestionan los alquileres, con atributos como cargo, oficina, turno y años de experiencia.
- **Gestión de alquileres:** Creación, seguimiento y cierre de contratos y actualización del estado del contrato. Cada alquiler se asocia a un cliente, un vehículo y un empleado responsable.

Modelado de la solución

Para que el sistema funcione correctamente en la empresa, se necesitan los siguientes perfiles:

- **Administrador del sistema:** Es la persona que instala el programa, lo mantiene actualizado, arregla posibles problemas y se encarga de que la base de datos funcione bien.
- **Empleado de oficina:** Es quien usa el programa a diario: crea los alquileres, da de alta nuevos clientes, consulta la disponibilidad de vehículos, etc.
- **Responsable de vehículos:** Atiende el estado de los coches y furgonetas (si están limpios, reparados, disponibles o alquilados) y actualiza esa información en el sistema.

Recursos hardware

Para ejecutar la aplicación se necesita un ordenador con acceso a un servidor donde esté instalada la base de datos MySQL.

Recursos software

El programa necesita:

- **Java JDK** (el kit de desarrollo de Java) para poder ejecutar la aplicación.
- **MySQL** como sistema gestor de la base de datos.
- **Eclipse o VS Code** para poder modificar el código si fuera necesario.
- **GitHub** para guardar versiones del proyecto y compartirlo entre los programadores.

2. EJECUCIÓN DE LA PRÁCTICA

Documentación técnica

- **App.java**: archivo principal que arranca el programa.
- **Conexion.java**: se encarga de conectar la aplicación con la base de datos.
- **Paquete DAO y DTO**: son dos carpetas dentro del proyecto que organizan el código. El DAO tiene las órdenes para leer y guardar datos en la base de datos. El DTO define cómo son los datos (qué información tiene un cliente, un vehículo, etc.).
- **Patrón DAO**: es una forma de programar que separa el acceso a los datos del resto de la lógica.
- **Streams y colecciones**: se usan estructuras como HashMap, TreeSet, ArrayList, LinkedList para guardar listas de datos en la memoria del ordenador. También se emplean streams (una forma moderna de recorrer listas) con operaciones como filtrar (filter), transformar (map), ordenar (sorted) y agrupar (collect).
- **Procedimientos MySQL**: dentro de la base de datos se han creado procedimientos almacenados que ayudan a obtener informes, como listar los alquileres activos o contar cuántos alquileres hubo cada mes.

Base de datos

Se ha creado con MySQL. Las tablas son: CLIENTE, CATEGORIA, VEHICULO, EMPLEADO, ALQUILER.

Aplicación Java

La estructura de carpetas del proyecto es:

RuedaFacil

```
├── src
├── App_Main
├── ConexionBD
├── DAO
│   ├── AlquilerD
│   ├── CategoriaD
│   ├── ClienteD
│   ├── EmpleadoD
│   └── VehiculoD
├── DTO
│   ├── Alquiler
│   ├── Categoria
│   ├── Cliente
│   ├── ClienteNoEncontradoException
│   ├── Empleado
│   ├── Vehiculo
│   └── VehiculoNoDisponibleException
```

Dentro de DAO está la lógica para acceder a los datos:

- **Gestión de clientes:** Crear nuevos, modificar datos, eliminar del sistema y consultar historial de alquileres.
- **Gestión de vehículos:** Registrar nuevos, modificar información, eliminar y consultar vehículos disponibles por categoría.
- **Gestión de alquileres:** Crear contratos, modificar datos, eliminar alquileres y gestión de estados del contrato.
- **Gestión de categorías y empleados:** Asociar vehículos con categorías, gestionar empleados.
- **Gestión de excepciones:** Se crean dos: `ClienteNoEncontradoException` (cuando se busca un cliente que no existe) y `VehiculoNoDisponibleException` (cuando se intenta alquilar un coche que ya está alquilado).

Características técnicas:

- Uso de `HashMap`, `TreeSet`, `ArrayList`, `LinkedList` y uso de streams `filter()`, `map()`, `sorted()`, `collect()`.
- `Comparable` y `Comparator` (para ordenar listas según varios criterios).
- Iteradores para eliminar elementos de forma segura.
- Excepciones personalizadas.

3. DOCUMENTACIÓN DEL SISTEMA

Manual de instalación y configuración

Requisitos previos:

- Tener instalado Java y MySQL (servidor de base de datos).
-

Pasos para instalar la base de datos:

- Abre el programa MySQL (por ejemplo, MySQL Workbench o la línea de comandos).
- Crea una nueva base de datos vacía con el nombre `RuedaFacil`.
- Ejecuta el script de creación (el listado de órdenes SQL que crea las tablas) que se incluye en el anexo de esta memoria. Eso construirá las tablas y las relaciones entre ellas.
- Ejecuta también el script de procedimientos (funciones, disparadores y cursores) para que la base de datos tenga todas las automatizaciones.

Configuración de la conexión:

- Dentro del código de Java, en el archivo `Conexion.java`, deberás escribir los datos de tu servidor MySQL: dirección (`localhost`), puerto, usuario y contraseña. La dirección suele ser `localhost:3306`.
- Compila y ejecuta
- Abre el proyecto en un entorno como Eclipse o VS Code.
- Asegúrate de tener añadido el conector de MySQL para Java (un archivo `.jar` que permite la conexión).
- Compila el programa y ejecuta la clase `App_Main`. Aparecerá el menú principal por consola.

Manual de usuario

Al arrancar el programa, verás una pantalla de texto con un menú numerado. Debes escribir el número de la opción que quieras y pulsar Enter. El menú principal ofrece acceso a: Clientes, Vehículos, Empleados, Alquileres y Salir.

Gestión de clientes: añadir, eliminar, modificar y ver

- Añadir cliente: El programa te pedirá el DNI, nombre, teléfono, correo y número de carnet. Introduce los datos y el cliente quedará guardado.
- Eliminar cliente: Introduce el DNI del cliente a borrar. El sistema te pedirá confirmación antes de eliminarlo.
- Modificar cliente: Introduce el DNI y luego los nuevos datos (puedes dejar los mismos si no quieres cambiarlos).
- Ver clientes: Muestra una lista con todos los clientes registrados.

Gestión de vehículos: añadir, eliminar, modificar estado, asignar categorías

- Añadir vehículo: Se pide matrícula, marca, modelo, año, combustible, plazas, precio por día y categoría (turismo, furgoneta, etc.).
- Eliminar vehículo: Mediante la matrícula.
- Modificar estado: Puedes cambiar el estado del vehículo (disponible, alquilado, en taller).
- Asignar categoría: Permite cambiar la categoría a la que pertenece un vehículo.

Gestión de empleados: registro, actualización, listado por criterios

- Registrar empleado: Se pide nombre, cargo (por ejemplo, Atención al cliente), oficina, turno (mañana/tarde) y años de experiencia.
- Actualizar empleado: modificar cualquiera de sus datos.
- Listar empleados: Puedes ordenarlos por cargo o por años de experiencia.

Gestión de alquileres: creación de un nuevo contrato, registro de una devolución, mensajes de error

- Nuevo contrato: El sistema te pedirá el DNI del cliente, la matrícula del vehículo, la fecha de inicio y la fecha prevista de devolución. Si el vehículo está disponible, se crea el alquiler y el vehículo pasa a estado "alquilado". Si no está disponible, aparecerá un mensaje de error (excepción de vehículo no disponible).
- Registrar devolución: Se introduce el ID del alquiler y la fecha real de devolución. El sistema calcula si ha habido retraso y actualiza el estado del contrato a "Completado". El vehículo vuelve a estar "disponible".
- Mensajes de error: Si intentas buscar un cliente que no existe, el sistema mostrará: "Cliente no encontrado". Si intentas alquilar un vehículo no disponible, avisará: "Vehículo no disponible en estos momentos".

4. CONCLUSIONES FINALES

El proyecto cumple con todos los objetivos que se plantearon al principio:

- CRUD completo (crear, leer, actualizar, borrar) para clientes, vehículos, empleados y alquileres.
- Se han utilizado colecciones y excepciones de Java para manejar los datos en memoria y los errores de forma ordenada.
- La estructura modular DAO/DTO permite que el programa sea fácil de entender y modificar en el futuro.

En resumen, la aplicación funciona cumpliendo con los requisitos mínimos exigidos en la práctica.

Propuestas de ampliación

Aunque el sistema ya cumple su cometido, se podrían añadir las siguientes mejoras en versiones futuras:

- Interfaz gráfica: Sustituir la actual interfaz por consola por una interfaz gráfica de usuario (con ventanas, botones y formularios) para que sea más intuitiva para los empleados de la oficina.
- Sistema de pagos: Integrar un sistema de facturación y una pasarela de pago (tarjeta de crédito, PayPal) para aceptar pagos online desde la propia aplicación.
- Aplicación móvil: Crear una versión para teléfonos móviles (Android/iOS) que permita a los clientes consultar su historial o hacer reservas.

5. BIBLIOGRAFÍA

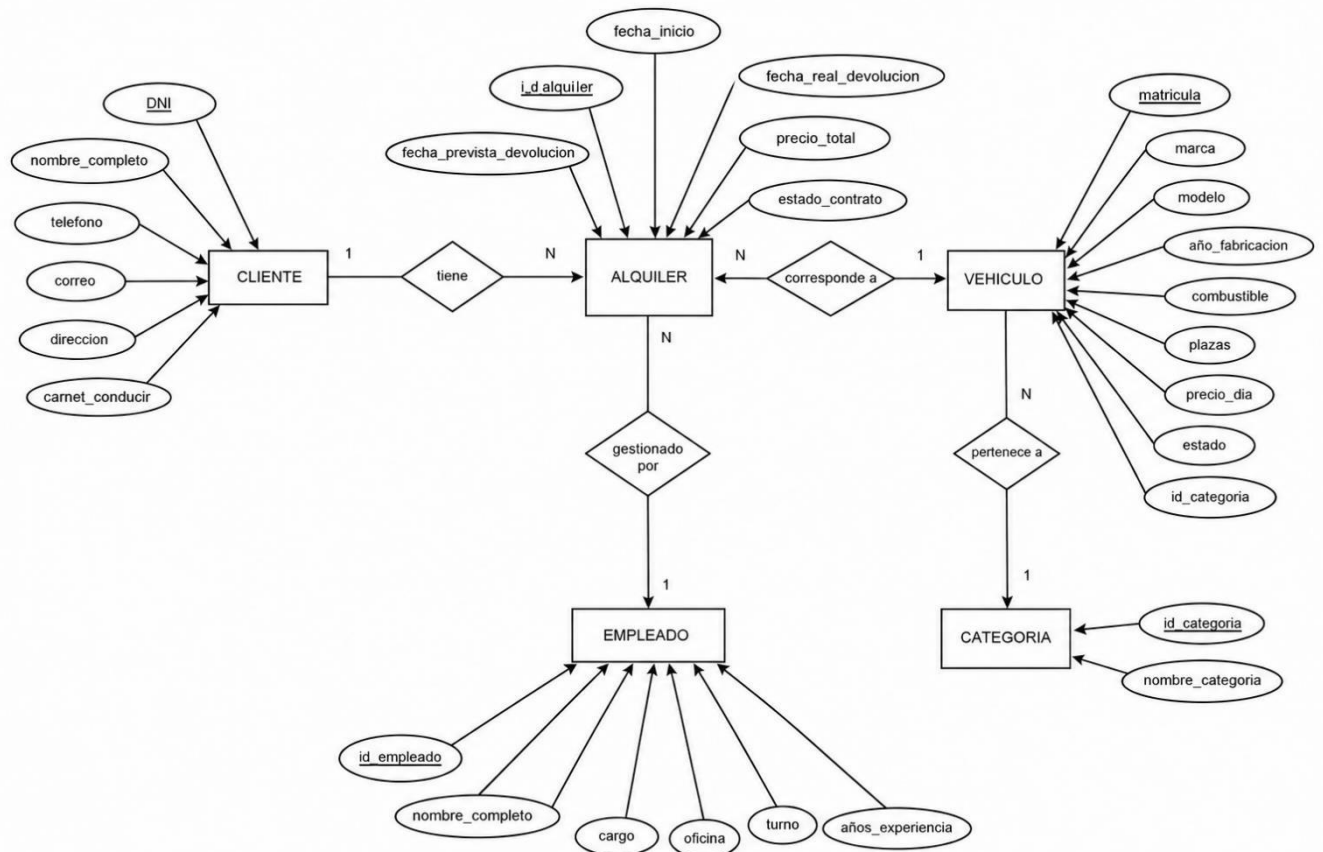
MySQL Tutorial: ejemplos de cursores y procedimientos almacenados.

Apuntes personales: temas 5 y 7 (procedimientos, funciones, triggers, cursores) y tema UT6_7_8 (colecciones en Java).

Documentación oficial de Java: Stream API, Comparable, Comparator, Iterator.

Asistencia por IA: Claude y Copilot se utilizaron para redactar documentación sobre colecciones, creación de procedimientos almacenados y resolución de errores en consola.

ANEXO – Base de Datos Modelo Entidad–Relación



Modelo Relacional:

CLIENTE: DNI (PK), nombre_completo, telefono, correo, direccion, carnet_conducir

CATEGORIA: id_categoria (PK), nombre_categoria

VEHICULO: matricula (PK), marca, modelo, año_fabricacion, combustible, plazas, precio_dia, estado, id_categoria (FK → CATEGORIA.id_categoria)

EMPLEADO: id_empleado (PK), nombre_completo, cargo, oficina, turno, años_experiencia

ALQUILER: id_alquiler (PK), fecha_inicio, fecha_prevista_devolucion, fecha_real_devolucion, precio_total, estado_contrato, DNI_cliente (FK → CLIENTE.DNI), matricula_vehiculo (FK → VEHICULO.matricula), id_empleado (FK → EMPLEADO.id_empleado)

Relaciones:

- Un cliente puede tener muchos alquileres → (1:N)
- Un vehículo puede aparecer en muchos alquileres → (1:N)
- Un empleado gestiona muchos alquileres → (1:N)
- Una categoría tiene muchos vehículos → (1:N)

Script de creación

```
CREATE DATABASE RuedaFacil;
```

```
USE RuedaFacil;
```

```
CREATE TABLE Categoria (  
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,  
    nombre_categoria VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Cliente (  
    dni VARCHAR(9) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    telefono VARCHAR(15) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    direccion VARCHAR(200),  
    num_carnet VARCHAR(20) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Empleado (  
    id_empleado INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    cargo ENUM (AtencionCliente, PreparadorVehiculo, Mecanico, Inspector, Director),  
    oficina VARCHAR(50),  
    turno VARCHAR(20),  
    anos_experiencia INT  
);
```

```
CREATE TABLE Vehiculo (  
    matricula VARCHAR(10) PRIMARY KEY,  
    marca VARCHAR(50) NOT NULL,  
    modelo VARCHAR(50) NOT NULL,  
    ano INT,  
    combustible VARCHAR(20),  
    plazas INT,
```

```

    precio_dia DECIMAL(10,2) NOT NULL,
    estado VARCHAR(20) DEFAULT 'disponible',
    id_categoria INT,
    FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria)
);

CREATE TABLE Alquiler (
    id_alquiler INT AUTO_INCREMENT PRIMARY KEY,
    fecha_inicio DATE NOT NULL,
    fecha_devolucion_prevista DATE NOT NULL,
    fecha_devolucion_real DATE,
    id_cliente VARCHAR(9) NOT NULL,
    id_empleado INT NOT NULL,
    matricula VARCHAR(10) NOT NULL,
    precio_total DECIMAL(10,2),
    estado_contrato ENUM (Activo, Completado, Cancelado),
    FOREIGN KEY (id_cliente) REFERENCES Cliente(dni),
    FOREIGN KEY (id_empleado) REFERENCES Empleado(id_empleado),
    FOREIGN KEY (matricula) REFERENCES Vehiculo(matricula)
);

INSERT INTO Categoria (nombre_categoria) VALUES
('turismo'),
('furgoneta'),
('motocicleta'),
('electrico'),
('todoterreno'),
('lujo');

INSERT INTO Cliente (dni, nombre, telefono, email, direccion, num_carnet) VALUES
('C11122233', 'Carlos Ruiz', '654321987', 'carlos.ruiz@example.com', 'Calle Independencia 789', '11122233C'),
('D44455566', 'Luisa Martínez', '655654321', 'luisa.martinez@example.com', 'Avenida Libertad 321', '44455566D'),
('E77788899', 'Javier Fernández', '656987654', 'javier.fernandez@example.com', 'Calle Principal 654', '77788899E');

INSERT INTO Empleado (nombre, cargo, oficina, turno, anos_experiencia) VALUES
('Carlos Martínez', 'Conductor', 'Oficina A', 'manana', 5),
('María López', 'Administrador', 'Oficina B', 'tarde', 3);

INSERT INTO Vehiculo (matricula, marca, modelo, ano, combustible, plazas, precio_dia, estado, id_categoria)
VALUES
('V001', 'Toyota', 'Corolla', 2018, 'gasolina', 5, 30.00, 'disponible', 1),
('V002', 'Volkswagen', 'Gol', 2019, 'diesel', 2, 25.00, 'disponible', 2),
('V003', 'Honda', 'CBR 600F', 2020, 'gasolina', 1, 50.00, 'disponible', 3),
('V004', 'Tesla', 'Model S', 2021, 'electrico', 2, 80.00, 'disponible', 4),
('V005', 'Jeep', 'Wrangler', 2017, 'gasolina', 4, 35.00, 'disponible', 5),
('V006', 'Lamborghini', 'Aventador', 2022, 'gasolina', 2, 150.00, 'disponible', 6);

INSERT INTO Alquiler (fecha_inicio, fecha_devolucion_prevista, id_cliente, id_empleado, matricula,
precio_total, estado_contrato) VALUES
('2023-04-01', '2023-04-07', 'C11122233', 1, 'V001', 150.00, 'activo'),

```



```
('2023-04-02', '2023-04-08', 'D44455566', 2, 'V002', 100.00, 'activo'),  
( '2023-04-03', '2023-04-09', 'C11122233', 1, 'V003', 50.00, 'activo'),  
( '2023-04-04', '2023-04-10', 'D44455566', 2, 'V004', 200.00, 'activo');
```

Procedimientos, Funciones, Triggers y Cursores

- **Procedimientos:**

obtener_alquileres_por_estado: Permite al gerente consultar rápidamente todos los alquileres según su estado (Activo, Completado o Cancelado).

```
DELIMITER //
```

```
CREATE PROCEDURE obtener_alquileres_por_estado(IN p_estado VARCHAR(20))
```

```
BEGIN
```

```
-- Si es válido, vacía la tabla auxiliar y la rellena con los alquileres filtrados.
```

```
IF p_estado NOT IN ('Activo','Completado','Cancelado') THEN
```

```
    SELECT 'El estado solo puede ser: Activo, Completado o Cancelado' AS mensaje; --
```

```
Si el estado no es válido, muestra un mensaje de error.
```

```
ELSE
```

```
    TRUNCATE TABLE estados_alquileres;
```

```
    INSERT INTO estados_alquileres(id_alquiler, fecha_inicio, fecha_devolucion_prevista,  
fecha_devolucion_real, estado_contrato, dni_cliente)
```

```
    SELECT id_alquiler, fecha_inicio, fecha_devolucion_prevista, fecha_devolucion_real,  
estado_contrato, id_cliente FROM Alquiler WHERE estado_contrato = p_estado;
```

```
END IF;
```

```
END //
```

```
DELIMITER ;
```

calcular_alquileres_por_mes: Genera un resumen mensual de alquileres para un año concreto.

```
DELIMITER //
```

```
CREATE PROCEDURE calcular_alquileres_por_mes(IN p_anno INT)
```

```
BEGIN
```

```
    DECLARE v_mes INT;
```

```
    TRUNCATE TABLE resumen_alquileres_mes;
```

```
-- Si el año no está en rango, devuelve un mensaje de error.
```

```
IF p_anno < 2000 OR p_anno > 2100 THEN
```

```
    SELECT 'Año fuera de rango (2000-2100)' AS mensaje;
```

```
ELSE
```

```
    SET v_mes = 1;
```

```
-- Recorre los 12 meses con un WHILE e inserta los totales en una tabla auxiliar.
```

```
    WHILE v_mes <= 12 DO
```

```
        INSERT INTO resumen_alquileres_mes(anno, mes, num_alquileres)
```

```
        SELECT p_anno, v_mes, COUNT(*) FROM Alquiler WHERE YEAR(fecha_inicio) = p_anno AND  
MONTH(fecha_inicio) = v_mes;
```

```
        SET v_mes = v_mes + 1;
```

```
    END WHILE;
```

```
END IF;
```

```
END //
DELIMITER ;
```

- **Funciones:**

mes_en_letras: Convierte un número de mes (1-12) en su nombre en español.

```
DELIMITER //
CREATE FUNCTION mes_en_letras(mes INT) RETURNS VARCHAR(10)
BEGIN
    IF mes < 1 OR mes > 12 THEN RETURN 'Error'; END IF;
    CASE mes
        WHEN 1 THEN RETURN 'Enero';
        WHEN 2 THEN RETURN 'Febrero';
        WHEN 3 THEN RETURN 'Marzo';
        WHEN 4 THEN RETURN 'Abril';
        WHEN 5 THEN RETURN 'Mayo';
        WHEN 6 THEN RETURN 'Junio';
        WHEN 7 THEN RETURN 'Julio';
        WHEN 8 THEN RETURN 'Agosto';
        WHEN 9 THEN RETURN 'Septiembre';
        WHEN 10 THEN RETURN 'Octubre';
        WHEN 11 THEN RETURN 'Noviembre';
        WHEN 12 THEN RETURN 'Diciembre';
    END CASE;
END //
DELIMITER ;
```

esBisiesto: calcula si un año es bisiesto o no

```
DELIMITER //
CREATE FUNCTION esBisiesto(anno INT)
RETURNS BOOLEAN
BEGIN
    DECLARE b BOOLEAN DEFAULT FALSE;
    IF anno <= 0 THEN RETURN FALSE;
    END IF;
    IF MOD(anno,4)=0 THEN
        SET b = TRUE;
        IF MOD(anno,100)=0 THEN SET b = FALSE;
        IF MOD(anno,400)=0 THEN SET b = TRUE;
        END IF;
    END IF;
    END IF;
    RETURN b;
END //
DELIMITER ;
```

estado_alquiler: Basado en las fechas del contrato.

```
DELIMITER //
CREATE FUNCTION estado_alquiler(p_id INT)
RETURNS VARCHAR(128)
BEGIN
    DECLARE f_prev DATE;
    DECLARE f_real DATE;
    DECLARE existe INT;
    SELECT COUNT(*) INTO existe FROM Alquiler WHERE id_alquiler = p_id; --
    Devuelve un mensaje indicando si el alquiler no existe.
    IF existe = 0 THEN RETURN 'El alquiler no existe'; END IF;
    SELECT fecha_devolucion_prevista, fecha_devolucion_real INTO f_prev, f_real FROM Alquiler WHERE
    id_alquiler = p_id;
    -- Devuelve un mensaje indicando si el alquiler no ha sido devuelto
    IF f_real IS NULL THEN RETURN 'Alquiler aún no devuelto'; END IF;
    -- Devuelve un mensaje indicando si el alquiler fue devuelto con retraso, puntualidad o
    adelanto IF f_real > f_prev THEN RETURN CONCAT('Devuelto con ', DATEDIFF(f_real,f_prev), '
    días de retraso');
    ELSEIF f_real = f_prev THEN RETURN 'Devuelto en la fecha prevista';
    ELSE
        RETURN CONCAT('Devuelto con ', DATEDIFF(f_prev,f_real), ' días de adelanto');
    END IF;
END //
DELIMITER ;
```

numero_alquileres: Devuelve el número de alquileres realizados en un mes y año concretos

```
DELIMITER //
CREATE FUNCTION numero_alquileres(mm INT, aaaa INT)
RETURNS INT
BEGIN
    DECLARE total INT;
    -- Devuelve el número de alquileres realizados en un mes y año concretos.
    IF mm NOT BETWEEN 1 AND 12 OR aaaa NOT BETWEEN 2000 AND 2100 THEN
        RETURN -1;
    -- Si los parámetros no son válidos, devuelve -1.
    END IF;
    SELECT COUNT(*) INTO total FROM Alquiler WHERE MONTH(fecha_inicio)=mm AND
    YEAR(fecha_inicio)=aaaa;
    RETURN total;
END //
DELIMITER ;
```

- **Triggers:** Cuando se crea un alquiler, el vehículo pasa automáticamente a estado 'alquilado'

DELIMITER //

CREATE TRIGGER tr_alquiler_insert_actualiza_vehiculo AFTER INSERT ON Alquiler

FOR EACH ROW

BEGIN

-- Cuando se crea un alquiler, el vehículo pasa automáticamente a estado 'alquilado'.

UPDATE Vehiculo SET estado = 'alquilado' WHERE matricula = NEW.matricula;

END //

DELIMITER ;

tr_alquiler_update_estado: actualiza el estado de un alquiler.

DELIMITER //

CREATE TRIGGER tr_alquiler_update_estado BEFORE UPDATE ON Alquiler

FOR EACH ROW

BEGIN

-- Si se registra una fecha de devolución real, el contrato pasa a 'Completado' y el vehículo vuelve a estar 'disponible'.

IF NEW.fecha_devolucion_real IS NOT NULL AND (OLD.fecha_devolucion_real IS NULL

OR OLD.fecha_devolucion_real <> NEW.fecha_devolucion_real) THEN SET NEW.estado_contrato = 'Completado';

UPDATE Vehiculo SET estado = 'disponible' WHERE matricula = NEW.matricula;

END IF;

END //

DELIMITER ;

- **Cursor: listar_vehiculos_alquilados:** Muestra todos los vehículos alquilados con todos sus datos.

DELIMITER //

CREATE PROCEDURE listar_vehiculos_alquilados()

BEGIN

DECLARE v_matricula VARCHAR(10);

DECLARE v_marca VARCHAR(50);

DECLARE v_modelo VARCHAR(50);

DECLARE v_cliente VARCHAR(100);

DECLARE fin_cursor INT DEFAULT 0;

DECLARE cur_vehiculos CURSOR FOR

SELECT v.matricula, v.marca, v.modelo, c.nombre FROM Vehiculo v JOIN Alquiler a ON v.matricula = a.matricula JOIN Cliente c ON a.id_cliente = c.dni WHERE a.estado_contrato = 'Activo';

-- Recorre todos los vehículos actualmente alquilados (estado Activo) y muestra su matrícula, marca, modelo y el cliente que lo tiene.

DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = 1;

OPEN cur_vehiculos; bucle: LOOP

FETCH cur_vehiculos INTO v_matricula, v_marca, v_modelo, v_cliente;

IF fin_cursor = 1 THEN LEAVE bucle; END IF;

```
        SELECT CONCAT('Vehículo alquilado: ', v_matricula, ' - ', v_marca, ' ', v_modelo, ' | Cliente: ',  
v_cliente) AS Informacion;    END LOOP bucle;  
    CLOSE cur_vehiculos;  
END //  
DELIMITER ;
```
